ArchivesSpace SWORD Deposits

Technical Specification

SWORD-based deposit of Archival Files from ArchivesSpace

Document Status: FINAL
Version: 0.12
Date: August 2026
Source Stories: A3 and A4
Project: LYRASIS Interoperability Project
Systems: ArchivesSpace SUI, ArchivesSpace PUI, DSpace REST API (7.x / DSpace 9.x contract), ArchivesSpace REST API

Purpose and Scope
Background
Assumptions
System Overview
Actors and Deployment Context
    Typical URL shapes
ArchivesSpace repositories/areas touched
Existing components to reuse or extend
    File Versions subrecord (the anchor)
    Resource/Archival Object → Create Digital Object modal (primary entry)
    Persistence path (Mode A)
    Host-Record Object children + create/link (Mode B)
    Multi-Digital-Object on one host record (Mode C)
    Configuration
Proposed new components
    Class responsibilities (sketch)
    File Version + Identifier population (client-side, default path)
    Mode B write path (server-side create + link)
    Mode C write path (multi-DO, in-form on the current AO)
Configuration model
    Tier 1 — System (plugin config.rb / AppConfig)
    Tier 2 — AS Administrator (per-repository SWORD settings)
Reference UI touchpoint
    Mode A — "Upload File Version" (host record: Resource or Archival Object)

- Mode B — "Upload and Link" panel (host record: Resource or Archival Object)
- Mode C — "Upload Digital Objects" button (host record: Resource or Archival Object)
- Accepted File Formats

Metadata and packaging
- Approach
- AS record fields written per mode
- Packaging mode behind the adapter

SWORD v2 protocol surface (implementation mapping)
- Client library note (sword2ruby)
- SWORD v3 forward compatibility

Data flows
- Mode A (single deposit, in-form population)
- Mode B (batch "Upload and Link")
- Data flow: Mode C (multi-DO "Upload Digital Objects")

Behavior Scenarios

Error Scenarios

Other Requirements
- Documentation
- Governance
- Error Reporting
- Performance and Scalability

Decisions: Implementation V1

Specification Gaps

Future / V2+ Implementation Considerations

Suggested epics

Related documents

Document history

Purpose and Scope

This specification defines requirements for a feature enabling ArchivesSpace staff users to deposit digital files from within an ArchivesSpace component record into a compliant repository via the SWORD (Simple Web-service Offering Repository Deposit) protocol, and then create ArchivesSpace digital object records linked to ArchivesSpace archival object records referencing the deposited content.

The spec defines **how ArchivesSpace would implement** a workflow in which a staff user deposits local files to a configured SWORD v2 repository and records the returned **public item URL(s)** on ArchivesSpace records. All three modes below share the same SWORD deposit

primitive (`DepositEntry`) and the same per-file granularity — one file → one repository item → one Digital Object — differing only in **which record the Digital Object links to** and **how it is persisted**.

Starting point — a Resource or an Archival Object (A-20): all three modes initiate from the **staff edit view of a Resource *or* an Archival Object**. Both record types expose the same **Instances** group (including "Add Digital Object → Create") and both sit on the same resource **tree**, so the modes work identically regardless of which one the staff user is editing. **Sandbox-confirmed 2026-07-20:** a Resource has the same Instances options as an AO, and a Resource in the top tree shows its AO children. The term **host record** below means "the Resource or Archival Object currently being edited." The only type-dependent detail is Mode B's notion of *immediate children*: for a **Resource** those are its **top-level Archival Objects**; for an **Archival Object** those are its **child Archival Objects**. The feature is **not** available on standalone Digital Object edit views, and **Digital Object Components are out of scope** (A-21 / D-17).

Where the returned URI is written (all modes, v0.6): each deposited file yields a Digital Object whose **Identifier** (`digital_object_id`) is set to the returned URI — this is a *required* field and holds the canonical/original location — **and** which carries a **File Version** whose `file_uri` is the same URI, so the ArchivesSpace PUI renders a clickable "view online" link (sandbox-verified 2026-07-20: an Identifier-only Digital Object shows the URI as plain text, whereas a File Version renders a clickable button). See A-19 and D-14.

Background

Staff managing digitized and born-digital archival collections in ArchivesSpace often need to deposit associated files into a SWORD-compliant repository for long-term access and discovery. The current workflow is entirely manual: files are uploaded to the repository, then URIs are copied back into ArchivesSpace digital object records one by one.

SWORD is a standard deposit protocol supported by DSpace (v2) and a growing number of other repositories. Integrating SWORD into ArchivesSpace would allow staff to initiate deposits without leaving the system they use to describe archival content, reducing errors and the time cost of maintaining links between the two systems.

Key distinctions from the companion ArchivesSpace-DSpace linking specification (A1-2):

1. A1-2 handles linking to pre-existing DSpace records; this spec handles the deposit of new content into DSpace where no DSpace record yet exists for the files being processed.
2. This spec covers pushing descriptive metadata from ArchivesSpace to DSpace with or without a deposit; A1-2 does not account for descriptive metadata, just bidirectional linking.

Assumptions

ID	Assumption
A-01	Target **ArchivesSpace 3.x+** deployed as the standard multi-app stack (staff SUI + JSON backend + PUI + Solr). Feature ships as an **ArchivesSpace plugin**.
A-02	**SWORD v2** is the only protocol implemented in v0.1. A `SwordProtocolAdapter` interface allows a future v3 adapter without rewriting orchestration.
A-03	The target repository exposes a **Service Document URL** and ≥1 **Collection** accepting the configured deposit types (binary bitstreams per `accepted_formats`; see Accepted File Formats).
A-04	**ArchivesSpace does not persist deposited binaries.** Files are streamed from the browser through the plugin to the SWORD endpoint and held only transiently. Only the returned **item URL** (and optional edit IRI) are written to AS.
A-05	Deposit orchestration runs in the **staff frontend plugin**, which already holds an authenticated backend session, so binaries never transit the JSON backend API.
A-06	**Granularity is 1:1:1**: one uploaded file → one repository item → one Digital Object. The returned URI is recorded on that Digital Object in **both** its required **Identifier** (`digital_object_id`) and one **File Version** `file_uri` (A-19), whether the Digital Object is pre-existing or created inline.
A-07	The `file_uri` written to AS is the repository's **public item URL** (e.g. `{dspaceBaseUrl}/handle/{handle}`), because PUI end users and staff both click it.
A-08	**Authentication (v2)** is **HTTP Basic Auth** with a service credential configured per AS repository (previously Gap G-01; per-user credentials deferred).
A-09	Deposit is **synchronous from the user's perspective** in v0.1. Even if the endpoint returns SWORD **In-Progress**, AS still writes the File Version immediately; staff manage visibility via existing **Publish?** and **Make Representative** controls.
A-10	**SWORD deposit metadata is a hard-coded AS → Dublin Core subset** read from the persisted **host record** (Resource or Archival Object; Mode B: the target child AO) and attached to the deposit (any configured format). This spec ships a **minimal code-based mapping** (at least `dc:title`); a fuller IR-specific mapping remains open (G-01). No Admin field-mapping UI for v1 implementation.

ID	Assumption
A-11	**The target Digital Object may not be persisted at deposit time.** In the primary Archival Object flow, the "Create Digital Object" modal holds an *unsaved* record (verified 2026-07-19: no ID/URI until "Create and Link"). Therefore the deposit **must not** depend on a saved Digital Object: it obtains the `file_uri` and **populates the in-memory File Version row**; AS persists everything on the record's normal Save / "Create and Link". SWORD DC metadata still comes from the **persisted host record** (A-12). The deposit control is **not** injected on the standalone Digital Object edit screen (A-21).
A-12	**Dublin Core for the SWORD package is sourced from the persisted host record** (Modes A/C: Resource or Archival Object being edited; Mode B: the **target child Archival Object**), via a **map of DC field → AS value** (`HostRecordDcMapper`). **Exception — Mode C `dc:title`:** because Mode C deposits many files onto one host record, **`dc:title` (and the AS Digital Object Title) use each file's basename without extension** so IR items and Digital Objects are not identically titled after the host (A-17). All **other** DC fields still come from the host record. Modes A/B take `dc:title` from the host/child record like the other fields.
A-13	**One binary per host record (Mode A/C) or per child (Mode B); the target records always pre-exist the deposit.** Staff create the Archival Objects first, then deposit a single file for each. In Mode A they iterate across the tree manually; in Mode B they populate many children in one action; in Mode C many files land on one host record. Each deposit is a single `DepositEntry`.
A-14	**Mode B writes server-side.** Because Mode B spans **sibling** records (many child Archival Objects) rather than one open form, it cannot rely on the native "Create and Link" form save. The plugin creates each Digital Object and links it to its child Archival Object via the **JSONModel API** (`create_digital_object` + append a digital-object instance to the child AO; AS has no PATCH → GET→merge→POST). This is the same server-side path A-11 marks optional for Mode A.
A-15	**Mode B lists only the host record's immediate (direct) children** and maps exactly **one file to one child** (1:1:1 preserved). For a **Resource** host, the immediate children are its **top-level Archival Objects**; for an **Archival Object** host, they are its **child Archival Objects** (A-20). Recursive/deep descendants and many-files-per-child are out of scope.
A-16	**Children are enumerated via ArchivesSpace's existing tree/children API** (e.g. the resource/archival-object tree endpoints the SUI already uses); the plugin does not maintain its own hierarchy.

ID	Assumption
A-17	**Mode C creates one Digital Object per file, all linked to the current Archival Object.** Each deposited file yields a distinct Digital Object with **Title = file name without extension**, **Identifier (`digital_object_id`) = returned URI**, and **one File Version** (`file_uri` = returned URI, A-19); a digital-object instance for each is added to the AO being edited. The same basename is used as SWORD **`dc:title`** for that file's deposit; other DC fields come from the host record (A-12). Multi-select is supported here (unlike Modes A/B) because the fan-out is *files → Digital Objects*, still 1:1 per file.
A-18	**Mode C persists on the host record's native Save.** Like Mode A (A-11), the plugin populates the open host-record edit form — a Resource or Archival Object (A-20) — with its Instances subrecords client-side; ArchivesSpace creates the new Digital Objects and their instance links when the staff user saves the host record. No plugin-side server write is required on the default path (contrast Mode B, A-14). *Whether the host-record form serializes multiple nested new Digital Objects on a single save should be verified against the sandbox before build (see D-13).*
A-19	**The returned URI is written to two Digital Object fields, in every mode.** (1) **Identifier** (`digital_object_id`) — a **required** field on Digital Objects (so it must be set at create time regardless), holding the canonical/original deposit URI; and (2) a **File Version** `file_uri` (public item URL, A-07) — because the ArchivesSpace PUI renders a clickable "view online" link only from a File Version, not from the Identifier (sandbox-verified 2026-07-20). File Versions therefore remain available for the *future* re-deposit / new-upload scenario (previously Gap G-10), while the initial deposit populates both fields.
A-20	**The starting point (host record) may be a Resource or an Archival Object — and only those two.** Both record types expose the same **Instances** subrecord group (with "Add Digital Object → Create") and share the resource **tree**, and both can carry digital-object instances — so all three modes work identically regardless of type. **Sandbox-confirmed 2026-07-20:** a Resource has the same Instances options as an AO, and a Resource in the top tree shows its AO children. The only type-dependent behavior is Mode B's *immediate children* (Resource → top-level AOs; AO → child AOs).
A-21	**The feature is confined to Resource and Archival Object edit views (D-17).** The "Create Digital Object" modal reuses the shared File Versions subrecord form that ArchivesSpace *also* renders on the standalone **Digital Object** edit screen; the plugin must therefore **scope its injection to the host-record context** (e.g. gate on the parent controller being `resources`/`archival_objects`, or on the File Versions form's DOM ancestry within a Resource/AO instances subrecord) so the

ID	Assumption
	deposit control does **not** appear when a Digital Object is edited directly. **Digital Object Components are out of scope**: no control, no create/link, no batch handling. Standalone Digital Object editing keeps its native "Add File Version" behavior unchanged.

System Overview

Systems: ArchivesSpace staff UI (SUI, Rails) and JSON backend API; ArchivesSpace PUI (display only); DSpace SWORD v2 endpoint (7.x/9.x contract); extensible to other SWORD-compliant repositories and SWORD v3.

Normative references:

- SWORD v2 Profile
- SWORD v2 specification
- swordapp/sword2ruby (candidate client; see maintenance note)
- ArchivesSpace API
- ArchivesSpace plugin development
- Companion designs: V1, `A1-2`
- Metadata mapping: `as-dc-mapping`

Mode	Entry point	Scope	Result per file	AS write path
A — Single deposit	The **host record** (Resource or AO) → Instances → Add Digital Object → **Create** modal	One file for the host record, repeated manually across records	The Digital Object being created gets **Identifier = URI** + a **File Version** (`file_uri` = URI)	**In-form population**, persisted on the modal's "Create and Link" (A-11)
B — Batch "Upload and Link"	A new panel on the **host record** (Resource or AO) listing its **immediate**	One file per child, all in a single action	One **Digital Object** (Identifier = URI + a File Version) linked to each child AO	**Server-side create + link** (the plugin creates a Digital Object and links it to each child)

Mode	Entry point	Scope	Result per file	AS write path
	children, one file input each			
C — "Upload Digital Objects"	A new button in the **Instances** section of the **host record** (Resource or AO), beside "Add Digital Object"	**Multiple files** for the **host record**, in one multi-select	One **Digital Object** per file (Title = filename without extension, Identifier = URI + a File Version), each linked to the host record	**In-form instance population**, persisted on the native **Save** (A-18)

Mode A is the primary, lowest-risk flow. Mode B populates many *children* quickly. Mode C populates many *Digital Objects on one* host record quickly. All three are `DepositBatch` = N × the same `DepositEntry` primitive (N = 1 for Mode A), and all three write the URI to **both** the Digital Object Identifier and a File Version (A-19). They differ only in write-back mechanics: Mode A and Mode C use **in-form population** persisted by a native Save; Mode B uses **server-side create + link** (see Mode B write path).

Mode A — single deposit (host record: Resource or Archival Object)

The staff user selects the **host record** — a Resource or an Archival Object — then opens **Instances** → **Add Digital Object** → **Create**, which renders the native **"Create Digital Object"** modal containing the standard **File Versions** subrecord form. The deposit control lives in that File Versions form. Although the same subrecord form is *also* reused on the standalone Digital Object edit screen (and on Digital Object Components), the plugin **scopes the control to the host-record context** so it renders only inside the Resource/AO Create Digital Object modal — it is deliberately **not** injected on the standalone `digital_objects` controller, and Digital Object Components are out of scope (A-21 / D-17). This costs one context guard rather than a global subrecord injection.

Observed workflow (from user studies; shown for a child Archival Object, identical on a Resource):

1. Staff user browses to a Resource or an Archival Object; the tree (upper panel) shows the hierarchy.
2. For the per-child pattern: if a needed child Archival Object does not exist, the staff user creates one per binary they intend to deposit (**records always exist before a deposit is attempted**), then returns to the host record.

3. Staff user clicks the **host record** (a Resource, or a child Archival Object) in the tree.
4. Staff user clicks **Instances → Add Digital Object → Create → "Upload File Version"** → the plugin deposits the single file via SWORD and populates the File Version's `file_uri` (and the Digital Object Identifier, A-19).
5. Staff user clicks **"Create and Link"**.
6. Staff user moves to the **next record** in the tree and repeats.

Mode A optimizes for **one binary per host record, quickly repeated** — not for attaching many binaries to a single Digital Object. Tree navigation is native ArchivesSpace behavior; Mode A adds no special multi-object mode.

Mode B — batch "Upload and Link" (host record: Resource or Archival Object)

To populate many children in one pass, the plugin adds an **"Upload and Link"** panel (proposed placement: a new section under **Instances** on the host record). It lists the host record's **immediate children**, each with its own file input. For a **Resource** the immediate children are its **top-level Archival Objects**; for an **Archival Object** they are its **child Archival Objects** (A-20).

Observed workflow (client ask):

1. Staff user browses to the host record (a **Resource** or a **parent Archival Object**); its children appear in the resource tree (upper panel) **and** as rows in the new "Upload and Link" section.
2. For each child they want to populate, the staff user selects a **local file** in that child's file input. (Children left blank are skipped.)
3. Staff user clicks a single **"Upload and Link"** button for the whole group.
4. For every child with a chosen file, the plugin **deposits the file via SWORD**, then **creates a Digital Object** (Identifier = returned URI, plus a single File Version `file_uri` = same URI — A-19) and **links it to that child Archival Object** as a digital-object instance.
5. A per-child result summary reports success/failure; failures are **fail-forward** (successful children are created and linked; failed children are listed for retry — previously Gap G-07).

Mode B still honors **1:1:1** (one file → one Digital Object with one File Version → one child AO) and **A-13** (children pre-exist). It does **not** attach multiple binaries to a single child, and it is **not** the deferred drag-and-drop wizard (see Out of scope).

Mode C — batch "Upload Digital Objects" (host record: Resource or Archival Object)

Mode C resembles the original v0.1 multi-select idea, but each selected file becomes its **own Digital Object** linked to the **host record** (Resource or AO) — not multiple File Versions on a

single Digital Object. The plugin adds an **"Upload Digital Objects"** button in the **Instances** section, beside the existing **"Add Digital Object"** button.

Observed workflow (client ask):

1. Staff user is on a **Resource** or **Archival Object** edit page (e.g. `…/resources/16/edit` or `…/resources/16/edit#tree::archival_object_1654`).
2. Staff user scrolls down to the **Instances** section.
3. Staff user clicks the **new "Upload Digital Objects"** button (next to "Add Digital Object").
4. A **multi-select** file dialog opens.
5. Staff user selects **one or more** files.
6. The plugin **deposits each file via SWORD** (Dublin Core from the **host record** via `HostRecordDcMapper`, with **`dc:title` overridden to that file's basename** — A-12 / A-17), then prepares **one new Digital Object per returned URI**. Each Digital Object has its **Title** set to the **file name without extension**, its **Identifier** (`digital_object_id`, required) set to the **returned URI**, and a **File Version** whose `file_uri` is the same URI (so the PUI shows a clickable link — A-19). Each is added to the host record as a digital-object instance.
7. Staff user clicks **"Save"** on the host record, which persists the new Digital Objects (with their File Versions) and their instance links.

Mode C preserves **one file → one repository item → one Digital Object** (still 1:1). It **reintroduces multi-select** — but only in the sense of *many files → many Digital Objects on one host record*; attaching many File Versions to a single Digital Object remains out of scope (each Digital Object gets exactly one File Version, mirroring its Identifier). Because the host record is the one open in the editor, persistence rides its native **Save** (like Mode A's in-form population, A-11/A-18).

In scope for v0.1–v1.0:

1. ArchivesSpace deployment/plugin context and where HTTP requests land
2. **Host record = a Resource or an Archival Object** (A-20): all three modes start from the staff edit view of either, since both expose the Instances group and the same tree
3. **Mode A** — entry from the **host record → Instances → Add Digital Object → Create** modal, on **Resource and Archival Object edit views only** (the control is scoped to that context — A-21 / D-17)
4. An **"Upload File Version"** control aligned with the existing **"Add File Version"** button, injected **only** where the File Versions subrecord form renders **within a Resource/AO Create Digital Object modal** (scoped away from the standalone Digital Object controller — A-21)

5. **Mode B** — an **"Upload and Link"** panel on the **host record** (Resource or AO) listing immediate children with one file input each, and a single action that deposits + creates + links a Digital Object per child (server-side)
6. **Mode C** — an **"Upload Digital Objects"** button in the Instances section of the **host record** (Resource or AO) that multi-selects files and creates one Digital Object per file (Title = filename, Identifier = returned URI, plus one File Version — A-19), linked to the host record and saved with it
7. The `DepositEntry` contract (one file → one repository item); `DepositBatch` = N × `DepositEntry`, realized by Modes B and C (fail-forward, previously Gap G-07)
8. **Hard-coded AS → Dublin Core** for deposits via `HostRecordDcMapper` (map of DC field → AS value from the host/child record; sample shows `dc:title`; Mode C overrides `dc:title` with file basename)
9. A version-abstracted SWORD client (v2 now; v3 adapter stub)
10. Two write-back paths: **in-form population** (Modes A and C) and **server-side create + link** (Mode B)
11. Configuration model at the repository level
12. Error handling mapped to ES01–ES04, including partial-batch failure for Modes B and C

Out of scope for v0.1–v0.10 (deferred or excluded):

- **Standalone Digital Object edit views as an entry point** (v0.8, D-17): the deposit controls are **not** shown on the `digital_objects` controller. Staff editing a Digital Object directly still get the native "Add File Version" only. The feature initiates exclusively from a Resource or Archival Object (A-20 / A-21).
- **Digital Object Components** are excluded entirely (v0.8, D-17): no deposit control, no create/link, no Mode B/C handling for DOCs.
- **Multiple File Versions on a single Digital Object from one multi-select** (de-prioritized: Mode C multi-select yields *one Digital Object per file*, not many File Versions on one Digital Object)
- The multi-page **deposit wizard** with **drag-and-drop** file→archival-object mapping previously described in BS02/BS03 (*explicitly deferred at client direction*). Mode B covers one file per immediate child; Mode C covers many files onto one AO; the free-form drag-drop mapping across arbitrary tree depth remains deferred, but reuses the same `DepositBatch` machinery.
- **Non-immediate descendants**: Mode B lists only the parent's **direct** children (A-15); deep/recursive selection is deferred.
- Admin-configurable or per-site AS → IR field-mapping UI (date conversion, multi-value flattening, etc.) — this spec ships a **hard-coded minimal** AS → Dublin Core subset for deposits (D-18); sites needing more edit the mapper in code
- Fuller IR-specific AS → DC mapping beyond the spec subset (**open**, pending ArchivesSpace + DSpace team input; see G-01)
- SWORD v3 implementation

- Re-deposit / update-in-place of previously deposited content (v2 Implementation Gap G-02)
- Changes to DSpace-side ingest workflow/approval configuration

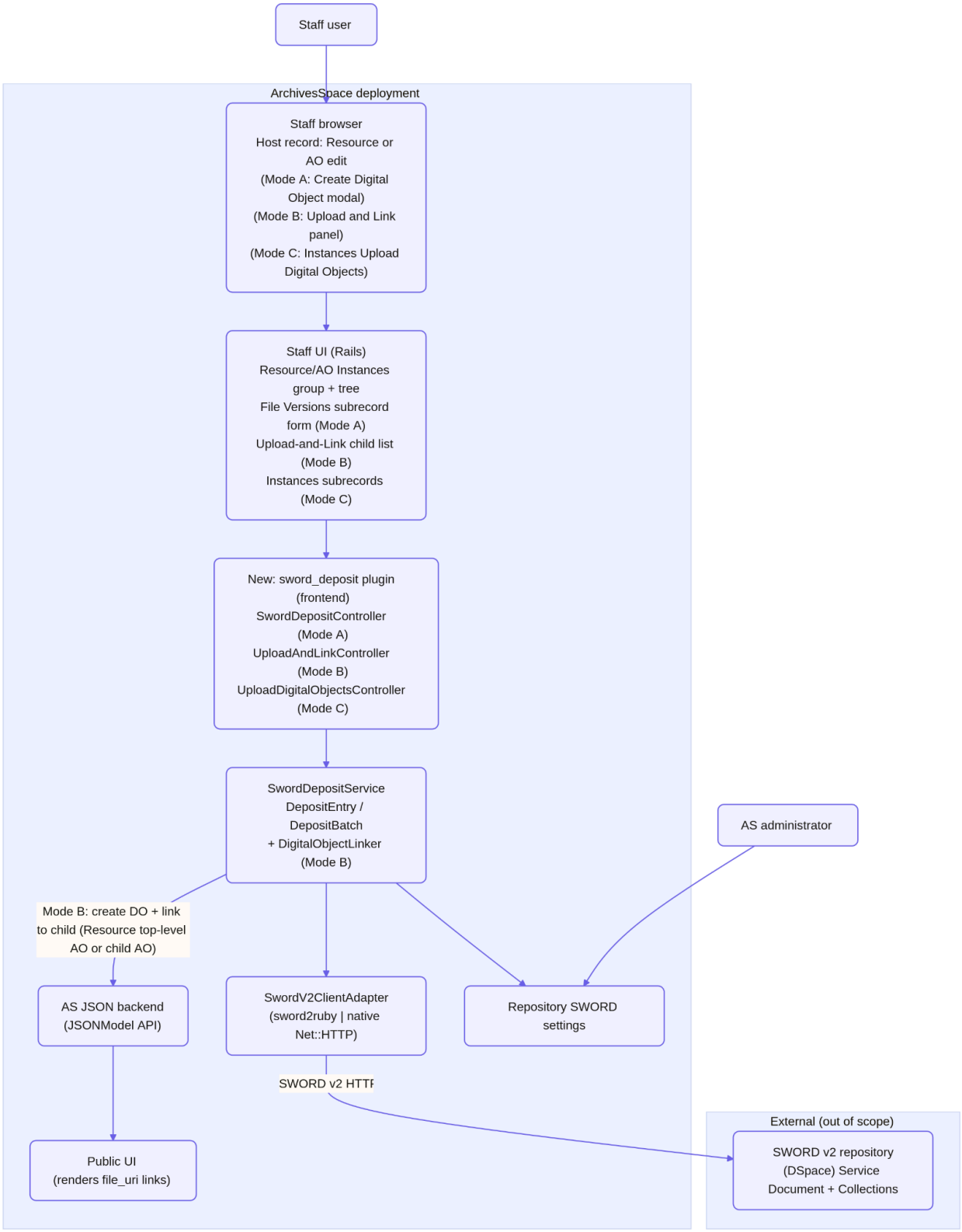
Naming constraint (program): the feature integrates with **any SWORD-compliant repository** via standard endpoints. DSpace is the reference target but must not be assumed beyond the protocol.

Design stance: implement the **atomic deposit primitive** (`DepositEntry`) — one file → one repository item → one returned URI. Every mode is `DepositBatch` = N × that primitive, differing only in where the URI is written:

- **Mode A** — N = 1; writes back a **File Version** in-form on the Digital Object modal.
- **Mode B** — N = children with a file; per entry, **server-side create** a Digital Object (with a File Version) and **link** it to a child AO.
- **Mode C** — N = selected files; per entry, prepare a **Digital Object** (Title = filename, Identifier = URI) as an **in-form instance** on the current AO, persisted on its native Save.

This mirrors the `LinkEntry`/`LinkBatch` and `create_digital_object` patterns from A1-2 — the difference from A1-2 is that the URI comes from a SWORD **deposit** rather than a link to a pre-existing DSpace item.

Actors and Deployment Context



Actor	Role
AS Administrator	Configures per-repository SWORD endpoint(s), credentials, default collection, protocol version, master enable (BS01).
AS Staff User (Mode A)	Selects the host record — a Resource or Archival Object — opens the Create Digital Object modal, chooses **one** local file and triggers deposit; reviews the resulting Identifier + File Version; sets Publish / Representative; saves via "Create and Link"; moves to the next record and repeats. (The control is not offered when editing a Digital Object directly — A-21.)
AS Staff User (Mode B)	On the host record (Resource or parent Archival Object), picks one local file per immediate child in the "Upload and Link" panel, clicks **"Upload and Link"** once, and reviews the per-child result summary.
AS Staff User (Mode C)	On the host record (Resource or Archival Object), clicks **"Upload Digital Objects,"** multi-selects files, reviews the Digital Objects populated into Instances (Title = filename, Identifier = URI, + a File Version), then clicks **Save**.
Repository/Collection manager	Configures SWORD permissions and collections on the target repository (outside AS).
PUI end user	Clicks the `file_uri` link to the deposited item (no deposit UI).

Typical URL shapes

Assume staff host `https://as.example.edu` (SUI):

Surface	Example	Notes
Resource edit (host record)	`/resources/16/edit`	Instances group + tree at the collection root; all three modes available here (A-20)
Archival Object edit (host record)	`/resources/16/edit#tree::archival_object_1646`	Same Instances group + tree at an AO node; all three modes available here

Surface	Example	Notes
Digital Object edit (existing)	`/digital_objects/27/edit`	**Out of scope (A-21 / D-17):** reuses the same File Versions form, but the plugin does **not** inject the deposit control here — native "Add File Version" only
Digital Object Component edit (existing)	`/digital_objects/27/edit#tree::digital_object_component_1`	**Out of scope (A-21 / D-17):** Digital Object Components are excluded from the feature entirely
Proposed upload endpoint (Mode A)	`POST /plugins/sword_deposit/deposit`	Multipart; params: repo id, **one** file, **host-record ref** (for AS → DC); server builds `dc_fields` via `HostRecordDcMapper`
Proposed batch endpoint (Mode B)	`POST /plugins/sword_deposit/deposit_and_link`	Multipart; params: repo id, host-record ref, and per-child `{ child_ao_ref → file }` pairs; per child, DC from that child AO; returns a per-child result report
Proposed multi-DO endpoint (Mode C)	`POST /plugins/sword_deposit/deposit_digital_objects`	Multipart; params: repo id, **many** files, host-record ref (shared non-title DC; **`dc:title` = per-file basename**); returns a per-file report of `{ title, identifier(file_uri), file_uri, status }` for in-form population
Children listing (existing tree API, Mode B)	Resource: `GET /repositories/:id/resources/:id/tree/root` (top-level AOs); AO: node/waypoint endpoints	Enumerate the host record's immediate children (A-16/A-20)

Surface	Example	Notes
Proposed config	`/plugins/sword_deposit/settings?repo_id=…`	Admin-only SWORD settings
Repository management (existing)	`/repositories`	Settings linked from here

ArchivesSpace repositories/areas touched

Area	Role in feature	Expected change level
New plugin `plugins/sword_deposit/`	Primary — controllers, service, SWORD adapter, config, JS, views, locales	**Major (new)**
Staff UI subrecord form for `file_version`	**Mode A:** add "Upload File Version" control beside "Add File Version", **scoped to the Resource/AO Create Digital Object modal** (not the standalone `digital_objects` controller — A-21); render resulting File Versions	**Moderate (via plugin override/partial + context guard)**
Staff UI **Resource / Archival Object edit** (Instances area)	Both record types share this UI (A-20). **Mode B:** inject the "Upload and Link" panel listing immediate children. **Mode C:** inject the "Upload Digital Objects" button beside "Add Digital Object"	**Moderate (via plugin partial/hook)**
`digital_object` JSONModel	All modes write **Identifier (`digital_object_id`, required) + one `file_version`** (A-19). **Mode A:** in-form on the DO being created. **Mode B:** `create_digital_object`	**None (data only)**

Area	Role in feature	Expected change level
	server-side. **Mode C:** one new DO per file via the host record form's nested instances on save. (`digital_object_component` is **not** touched — A-21.)	
`resource` / `archival_object` JSONModel	**Mode B:** append a digital-object `instances[]` entry linking the new Digital Object to each child AO (server-side). **Mode C:** append **N** digital-object `instances[]` entries to the host record (Resource or AO), persisted on Save	**None (data only)**
Resource/AO **tree API**	**Mode B:** enumerate the host record's immediate children — top-level AOs for a Resource, child AOs for an AO (A-16/A-20)	**None (read only)**
Repository configuration UI	Host SWORD Deposit Settings section	**Minor (plugin-provided)**
SWORD client dependency	`sword2ruby` gem *or* vendored native client	**Dependency**
PUI	Renders `file_versions[].file_uri` (existing behavior)	**None**

No core AS fork required: everything is achievable through the plugin mechanism (frontend controllers/assets/views, optional backend model for config, `config.rb`).

Existing components to reuse or extend

ArchivesSpace has **no SWORD code today**. Closest reusable machinery:

File Versions subrecord (the anchor)

Component	Reuse
`file_version` subrecord form partial (staff UI)	The section rendered under **File Versions**; "Add File Version" is its subrecord-add control. The new control lives here **but only when the form renders inside a Resource/AO Create Digital Object modal** — the plugin gates on host-record context so it is absent when a Digital Object (or Component) is edited directly (A-21).
Subrecord add JS (`add_subrecord` / `subrecord.js` patterns)	Precedent for appending a File Version row client-side; the upload flow appends **populated** rows (with `file_uri`) instead of empty ones.
`is_representative` / "Make Representative"	Left to the staff user post-deposit (A-09).
`publish` checkbox	Default `true` on deposited File Versions; staff can toggle (A-09).

Sandbox-verified File Version fields (2026-07-19): `Make Representative`, **File URI**, `Publish?`, `Use Statement`, `XLink Actuate Attribute`, `XLink Show Attribute`, `File Format Name`, `File Format Version`, `File Size (Bytes)`, `Checksum`, `Checksum Method`, `Caption`. The deposit populates at least **File URI** and `Publish?`; other fields remain staff-editable.

Resource/Archival Object → Create Digital Object modal (primary entry)

Both Resources and Archival Objects expose the identical Instances group and "Add Digital Object → Create" modal (A-20), so this reuse applies to either host record.

Component	Reuse
Resource / Archival Object **Instances** subrecord (`Add Digital Object` → dropdown → `Create`)	Native control that opens the **"Create Digital Object"** modal; the deposit control appears inside that modal's File Versions form.

Component	Reuse
"Create Digital Object" modal (embedded, unsaved record)	**Verified (on an AO):** the Digital Object is unsaved (no ID/URI) and is created + linked to the host record only on **"Create and Link"**. The File Versions subrecord renders **inline** here, identical to the standalone form. The Resource modal is the same component (A-20).
`add_form_and_link` / nested subrecord persistence	The whole nested record (Digital Object + File Versions + instance link) is persisted by AS on "Create and Link" — the plugin does **not** write it server-side.

Persistence path (Mode A)

Component	Reuse
Native subrecord form submit ("Create and Link" / Save)	**Default write path (A-11):** the deposited File Version row is populated in the form and persisted by AS's own save. Works whether the Digital Object pre-exists or is being created inline. No plugin-side JSONModel write required.
`JSONModel(:digital_object)` (`find → append → save`)	**Optional path only** for a *pre-existing, already-saved* Digital Object if immediate auto-save is later chosen (D-03). Not usable in the unsaved AO-create modal. AS has no PATCH; this is the GET→merge→POST equivalent. (Digital Object Components are excluded — A-21.)
Frontend authenticated backend session	Plugin controller reuses the staff session's backend token — no separate auth.

Host-Record Object children + create/link (Mode B)

Component	Reuse
Resource / Archival Object **tree API**	List the host record's immediate children (title + ref) to render the "Upload and Link" rows — top-level AOs for a Resource, child AOs for an AO (A-16/A-20).
`create_digital_object` (A1-2 precedent)	For each child with a file: `POST /repositories/:id/digital_objects` with **Identifier (`digital_object_id`) = the SWORD item URL** (required) and a single `file_version` whose `file_uri` is the same URL (A-19).
Digital-object **instance** on `archival_object`	`GET` the child AO → append `instances[]` a digital-object instance referencing the new DO → `POST` (full JSONModel; GET→merge→POST). Mirrors A1-2 `create_digital_object` linking.
Frontend authenticated backend session	Same session/token reused for all reads and writes.

Multi-Digital-Object on one host record (Mode C)

Component	Reuse
Resource / Archival Object **Instances** subrecord + "Add Digital Object"	The new "Upload Digital Objects" button sits beside it (same group on both record types, A-20). Each deposited file adds one **digital-object instance** row (like the native "Create Digital Object" flow, but repeated per file and pre-filled).
`add_form_and_link` / nested subrecord persistence	Same mechanism Mode A relies on: AS creates the nested new Digital Object(s) and links them when the host record is saved — extended from one to **N** instances (A-18; verify multi-nesting, D-13).

Component	Reuse
Native **Save** submit (Resource or AO)	Default write path (A-18). Populated in-form, persisted by AS's own save — no plugin-side server write on the default path.
Digital Object `title` / `digital_object_id` / `file_versions[]`	Set per file: **Title = filename without extension**, **Identifier = returned URI** (required), **plus one File Version** (`file_uri` = returned URI) for PUI clickability (A-19).

Configuration

Component	Reuse
Plugin `config.rb` / `AppConfig`	Global feature flags/limits.
Repository-scoped settings (plugin-managed store)	Per-repository endpoint records (see Configuration Fields table).

Proposed new components

Plugin layout (ArchivesSpace conventions):

```
None
plugins/sword_deposit/
  config.rb                              # AppConfig defaults, feature flag
  frontend/
    controllers/
      sword_deposit_controller.rb        # Mode A: POST
/plugins/sword_deposit/deposit (multipart)
      upload_and_link_controller.rb      # Mode B: POST
/plugins/sword_deposit/deposit_and_link
      sword_settings_controller.rb       # repository SWORD settings CRUD + Test
connection
    views/
      sword_deposit/_upload_button.html.erb        # Mode A: "Upload File Version"
control
```

```
      sword_deposit/_upload_and_link_panel.html.erb # Mode B: child list + file
inputs + button
      sword_settings/index.html.erb
    assets/
      sword_deposit.js                  # Mode A: single-file input, populate
File Version
      sword_upload_and_link.js          # Mode B: per-child inputs, submit,
per-child results
    locales/en.yml                      # button labels, errors, tooltips
    plugin_init.rb                      # inject Mode A button + Mode B panel
  backend/
    model/
      sword_endpoint_config.rb          # per-repo endpoint persistence (or repo
preference)
    controllers/
      sword_config_controller.rb        # optional backend API for config (if not
frontend-only)
  lib/
    sword_deposit_service.rb            # DepositEntry / DepositBatch
orchestration
    digital_object_linker.rb            # Mode B: create_digital_object + link
instance to child AO
    archival_object_children.rb         # Mode B: enumerate parent's immediate
children (tree API)
    metadata/
wrapper
      host_record_dc_mapper.rb          # hard-coded AS host/child record →
Dublin Core map (D-18)
      deposit_package.rb                # binary bitstream + Atom/DC EntryPart
wrapper
    sword/
      sword_protocol_adapter.rb         # interface: service_document, deposit,
status
      sword_v2_client_adapter.rb        # wraps sword2ruby OR native Net::HTTP
      sword_v3_client_adapter.rb        # stub -> NotImplemented
      sword_adapter_factory.rb          # select by configured version
      deposit_result.rb                 # item_url, edit_iri, in_progress, status
    sword_deposit_audit_log.rb          # timestamp, user, repo, collection,
result
```

Class responsibilities (sketch)

`HostRecordDcMapper.from(host_ref)` —
hard-coded AS host/child record → Dublin Core map for deposits (D-18). Sample shows **title only**; other fields from the AS → Dublin Core mapping table follow the same pattern. Omit a DC element when the AS source value is absent. **Mode C** starts from this map, then **overrides `dc:title`** with each file's basename (A-17) so multi-file deposits are not identically titled.

```
None
# Returns a map of DC field => AS metadata value, e.g. { "dc:title" => "...",
"dc:creator" => "...", ... }
# Modes A/C: host_ref is the Resource or Archival Object being edited.
# Mode B: host_ref is the target child Archival Object.
def self.from(host_ref)
  ref    = JSONModel.parse_reference(host_ref)
  record = JSONModel(ref[:type]).find(ref[:id])   # :resource or :archival_object
  dc     = {}

  # Sample field — Modes A/B use this host/child title as dc:title.
  # Mode C overwrites dc:title per file with File.basename(filename, ".*") after
calling this
  # (see deposit_digital_objects_batch); other DC keys from this map are kept.
  title = record['title'].presence || record['display_string']
  dc['dc:title'] = title if title.present?

  # Other fields from the AS → DC table (creator, date, description, subject, …)
  # follow the same pattern: read AS, write dc['dc:…'] when present.
  dc
end
```

`SwordDepositController#deposit` — staff HTTP entry (multipart). Extracts DC from the host record, then deposits:

```
None
# POST /plugins/sword_deposit/deposit  (multipart)
# params: repo_id, collection_href, file,   # ONE file (A-13)
#         host_ref                           # persisted Resource or Archival
Object (A-12 / A-20)

def deposit
  cfg       = SwordEndpointConfig.for_repository(params[:repo_id])   # enabled?
(ES01)
```

```
  dc_fields = HostRecordDcMapper.from(params[:host_ref])            # AS → DC map
(title sample above)
  result    = SwordDepositService.new(cfg, current_backend_session)
                .deposit_entry(params[:file],
                               dc_fields: dc_fields,
                               collection_href: params[:collection_href])
  render json: result   # { filename, status, file_uri, error } -> JS fills one
row
end
```

`SwordDepositService` — the contract everything hangs off. Note it returns the `file_uri` and does **not** write to AS (A-11): persistence is the native form save. `dc_fields` is the Dublin Core map attached to the deposit.

```
None
# DepositEntry: one file -> one item -> one file_version row (1:1:1)
def deposit_entry(file, dc_fields:, collection_href:)
  package = DepositPackage.build(file, dc_fields: dc_fields)   # binary + Atom
EntryPart DC (D-18)
  res     = adapter.deposit(collection_href || cfg.default_collection, package,
cfg) # SWORD POST
  audit_log.record(file, cfg, collection_href, dc_fields, res)
  DepositResult.ok(file.original_filename, res.item_url)   # public handle URL
(A-07)
ensure
  package&.dispose                                          # drop transient bytes
(A-04)
@@ -397,15 +421,27 @@ end
# DepositBatch: retained ONLY for the deferred wizard (batch across archival
objects).
# The shipping "Upload File Version" control deposits one file per child Archival
Object,
# so it calls deposit_entry directly. N x DepositEntry, fail-forward (previously
Gap G-07).
def deposit_batch(files, dc_fields:, collection_href:)
  DepositReport.new(files.map { |f|
    begin;  deposit_entry(f, dc_fields: dc_fields, collection_href:
collection_href)
    rescue => e; DepositResult.fail(f.original_filename, e); end
  })
end
```

`DepositPackage.build` — builds the SWORD deposit with Dublin Core from the map (title sample; other DC keys follow the same pattern):

```
None
def self.build(file, dc_fields:)
  # Content-Type from the file; Content-Disposition filename; optional Slug /
In-Progress.
  # Attach Atom metadata from dc_fields — sample shows title only:
  #   if (title = dc_fields['dc:title']).present?
  #     entry.add_dublin_core('title', title)   # sword2ruby-style or native Atom
builder
  #   end
  # Native Net::HTTP and sword2ruby both can carry an Atom entry alongside the
binary;
end
```

`SwordV2ClientAdapter` — thin, library-agnostic:

```
None
class SwordV2ClientAdapter          # implements SwordProtocolAdapter
  def service_document(cfg); end    # GET Service-Document IRI (Basic auth)
  def deposit(collection_href, package, cfg)
    # POST binary to Collection IRI with headers:
    #   Authorization: Basic ..., Content-Type, Content-Disposition (filename),
    #   optional Slug, In-Progress. Parse DepositReceipt -> DepositResult.
  end
  def status(edit_iri, cfg); end    # optional in v0.1 (In-Progress polling)
end
```

File Version + Identifier population (client-side, default path)

Create Digital Object modal (opened from a Resource or Archival Object) with **both** targets (A-19): it sets the Digital Object **Identifier** (`digital_object_id`, a required field — the canonical URI) **and injects a populated File Version row** (the same DOM the native "Add File Version" button drives). AS persists both on "Create and Link" (A-11). No plugin-side JSONModel write is needed. The handler is wired **only** in this host-record modal context — not on the standalone Digital Object edit screen (A-21).

```
None
// sword_deposit.js (sketch): after POST /plugins/sword_deposit/deposit (one file)
function onDepositResult(r) {
  if (r.status !== "ok") { showRowError(r.filename, r.error); return; } //
ES02/03/04
  setDigitalObjectIdentifier(r.file_uri);  // required field; canonical URI (A-19)
  var $row = addFileVersionRow();          // reuse AS subrecord add
  $row.find("[name$='[file_uri]']").val(r.file_uri);   // public handle URL -> PUI
link (A-07)
  $row.find("[name$='[publish]']").prop("checked", true); // A-09; staff can
toggle
  // is_representative left unchecked -> staff uses "Make Representative"
}
// persistence happens on the native "Create and Link" submit
// Note (A-21): this handler is wired only inside the Resource/AO Create Digital
Object modal.
// It is not bound on the standalone digital_objects edit view, and Digital Object
// Components are out of scope entirely.
```

Optional immediate-save path (pre-existing Digital Object only, see D-03): if a future iteration wants deposits to persist without waiting for Save on an *already-saved* record:

```
None
rec = JSONModel(target_type).find(id_from(target_ref))   # only
when DO already exists
rec.file_versions << {
  "jsonmodel_type"          => "file_version",
  "file_uri"                => item_public_url,   # A-07
  "publish"                 => true,              # A-09; staff can
toggle
  "is_representative"       => false,             # staff uses
"Make Representative"
  "xlink_actuate_attribute" => "onRequest",
  "xlink_show_attribute"    => "new"
}
rec.save                                          # full JSONModel
POST (no PATCH)
```

Mode B write path (server-side create + link)

Mode B spans **sibling** child Archival Objects (of a Resource or a parent AO host record), so there is no single open form to populate; the plugin writes to AS itself (A-14). For each child that has a chosen file it runs one `DepositEntry` (SWORD deposit → `file_uri`), then creates a Digital Object and links it to that child. Failures are **fail-forward** and reported per child (Previously Gap G-07).

`UploadAndLinkController#deposit_and_link` — Mode B HTTP entry (multipart):

```
None
# POST /plugins/sword_deposit/deposit_and_link  (multipart)
# params: repo_id, parent_ao_ref, collection_href,
#         children: [ { child_ao_ref:, file: }, ... ]   # one file per child
(A-15)
def deposit_and_link
  cfg    = SwordEndpointConfig.for_repository(params[:repo_id])   # enabled?
(ES01)
  report = SwordDepositService.new(cfg, current_backend_session)
             .deposit_and_link_batch(params[:children],
                                     collection_href: params[:collection_href])
  render json: report   # per-child { child_ao_ref, status, digital_object_uri,
file_uri, error }
end
```

`SwordDepositService#deposit_and_link_batch` — `DepositBatch` + create/link:

```
None
def deposit_and_link_batch(children, collection_href:)
  DepositReport.new(children.map { |c|
    begin
      dc_fields = HostRecordDcMapper.from(c[:child_ao_ref]) # DC from the child AO
(A-12)
      res = deposit_entry(c[:file], dc_fields: dc_fields,
                          collection_href: collection_href) # SWORD deposit
(shared primitive)
      link = DigitalObjectLinker.new(session).create_and_link(
               child_ao_ref: c[:child_ao_ref], file_uri: res.file_uri)     #
create DO + instance link
      DepositResult.linked(c[:child_ao_ref], link.digital_object_uri,
res.file_uri)
    rescue => e
```

```
      DepositResult.fail(c[:child_ao_ref], e)   # fail-forward; other children
continue (Previously Gap G-07)
    end
  })
end
```

`DigitalObjectLinker#create_and_link` — the AS write (A-14; mirrors A1-2 `create_digital_object`):

```
None
def create_and_link(child_ao_ref:, file_uri:)
  child = JSONModel(:archival_object).find(id_from(child_ao_ref))
  do_obj = JSONModel(:digital_object).new._always_valid!
  do_obj.title = child.display_string                 # AS Digital Object Title
(write-back); SWORD dc:title is separate (HostRecordDcMapper)
  do_obj.digital_object_id = file_uri                 # Identifier = returned URI
(A-19)

  do_obj.file_versions = [{
    "jsonmodel_type" => "file_version", "file_uri" => file_uri,          # A-07
    "publish" => true, "is_representative" => false,                     # A-09
    "xlink_actuate_attribute" => "onRequest", "xlink_show_attribute" => "new"
  }]
  created = do_obj.save                                  # POST
/repositories/:id/digital_objects
  child.instances << {                                   # GET->merge->POST (no
PATCH)
    "jsonmodel_type" => "instance", "instance_type" => "digital_object",
    "digital_object" => { "ref" => created.uri }
  }
  child.save
  OpenStruct.new(digital_object_uri: created.uri)
end
```

> **Partial state (ES02/ES05):** if the SWORD deposit succeeds but the AS create/link fails, a repository item exists with no AS link. The failure is reported for that child and the **Edit-IRI is logged** for cleanup (same caveat as A1-2 "DSpace PATCH fails after AS success"). Default policy is fail-forward, not rollback (see D-12).

Mode C write path (multi-DO, in-form on the current AO)

Mode C is `DepositBatch` (many files) but writes like Mode A: it **deposits each file**, then returns a per-file descriptor the browser uses to **populate a digital-object instance** on the open host-record form (Resource or AO). AS creates the new Digital Objects and links them when the staff user clicks **Save** on the host record (A-18). No plugin-side server write is required on the default path.

`UploadDigitalObjectsController#deposit_digital_objects` — Mode C HTTP entry (multipart):

```
None
# POST /plugins/sword_deposit/deposit_digital_objects  (multipart)
# params: repo_id, collection_href, files[],   # MANY files (A-17); host record is
the open editor record
#         host_ref                              # current Resource or Archival
Object (audit/metadata context; A-20)
def deposit_digital_objects
  cfg    = SwordEndpointConfig.for_repository(params[:repo_id])   # enabled?
(ES01)
  report = SwordDepositService.new(cfg, current_backend_session)
             .deposit_digital_objects_batch(params[:files], host_ref:
params[:host_ref],
                                            collection_href:
params[:collection_href])
  render json: report   # per-file { filename, title, identifier(=file_uri),
file_uri, status, error }
end
```

`SwordDepositService#deposit_digital_objects_batch` — deposit each file, return DO descriptors (no AS write; A-18). Non-title DC comes from the **host record** once; each file sets **`dc:title`** **and the AS Digital Object Title** to that file's basename (A-17).

```
None
def deposit_digital_objects_batch(files, host_ref:, collection_href:)
  dc_base = HostRecordDcMapper.from(host_ref)               # shared host-record
DC (A-12)
  DepositReport.new(files.map { |f|
    begin
      title = File.basename(f.original_filename, ".*")      # per-file title
(A-17)
```

```
      dc_fields = dc_base.merge("dc:title" => title)         # override host title
for this file
      res  = deposit_entry(f, dc_fields: dc_fields,
                           collection_href: collection_href) # SWORD deposit
(shared primitive)
      DigitalObjectInstanceBuilder.descriptor(
        filename: f.original_filename, title: title, file_uri: res.file_uri)
    rescue => e
      DepositResult.fail(f.original_filename, e)   # fail-forward; other files
continue (Previously Gap G-07)
    end
  })
end
```

`DigitalObjectInstanceBuilder.descriptor` — basename→**AS** Digital Object Title **and** SWORD **`dc:title`** (Mode C); URI→both Identifier and File Version (A-19):

```
None
def self.descriptor(filename:, title:, file_uri:)
  { "status"     => "ok",
    "filename"   => filename,
    "title"      => title,       # file name without extension (A-17)
    "identifier" => file_uri,    # -> digital_object_id (required, canonical URI)
    "file_uri"   => file_uri }   # -> a File Version (PUI clickable link)
(A-07/A-19)
end
```

Mode C in-form population (client-side): for each successful descriptor, the browser adds a **digital-object instance** to the host record's Instances subrecord — creating a nested new Digital Object with `title`, `digital_object_id`, **and one File Version** pre-filled — exactly as if the staff user had used "Add Digital Object → Create" once per file. AS persists all of them on the host record's native **Save** (A-18).

```
None
// sword_upload_digital_objects.js (sketch): after POST
.../deposit_digital_objects (many files)
function onDepositResults(report) {
  report.forEach(function (r) {
    if (r.status !== "ok") { showFileError(r.filename, r.error); return; } //
ES02/03/04
```

```
    var $inst = addDigitalObjectInstance();                // reuse AS "Add
Digital Object" (nested create)
    $inst.find("[name$='[digital_object][title]']").val(r.title);           //
filename w/o extension
    $inst.find("[name$='[digital_object][digital_object_id]']").val(r.identifier);
// required, canonical URI
    var $fv = addFileVersionRow($inst);                    // one File Version on
the nested DO (A-19)
    $fv.find("[name$='[file_uri]']").val(r.file_uri);       // public URL -> PUI
clickable link (A-07)
    $fv.find("[name$='[publish]']").prop("checked", true);  // A-09; staff can
toggle
  });
  // persistence happens on the host record's native Save submit (A-18)
}
```

Verify before build (D-13): Mode A confirmed that *one* nested Digital Object is created + linked on save. Mode C relies on the host-record form (Resource or AO) serializing **multiple** nested new Digital Objects on a single Save. If the sandbox shows that only one nested create is supported per save, Mode C falls back to the **server-side pre-create** path (like Mode B's `DigitalObjectLinker`, one DO per file, then link) — at the cost of possible orphaned items if the save is abandoned (ES05).

None

Configuration model

Two tiers, matching the Configuration Fields table and AS conventions.

Tier 1 — System (plugin `config.rb` / `AppConfig`)

Key	Example	Purpose
`AppConfig[:sword_deposit_enabled]`	`true`	Master switch; hides control, rejects endpoint when false (ES01)
`AppConfig[:sword_deposit_max_upload_bytes]`	`104857600`	Per-file upload cap
`AppConfig[:sword_deposit_allowed_mime_types]`	`["application/pdf", “application/zip”]`	Default allow-list when a repository leaves `accepted_formats` blank; See Accepted File Formats
`AppConfig[:sword_deposit_client]`	`native`	`native` (Net::HTTP) or `sword2ruby`
`AppConfig[:sword_deposit_default_protocol]`	`v2`	Adapter default

Tier 2 — AS Administrator (per-repository SWORD settings)

Reached via **Repository management → SWORD Deposit Settings**:

Field Group: SWORD Settings

Field	Type	Required	Description
`display_name`	string	No	Label when multiple endpoints exist
`enabled`	boolean	Yes	Per-repository master switch (drives control visibility, ES01)
`service_document_url`	URL	Yes	SWORD v2 Service Document URL

Field	Type	Required	Description
`protocol_version`	enum	Yes	`v2` (default); `v3` disabled until implemented
`auth_type`	enum	Yes	`basic` (v0.1); schema allows future OAuth/API key (Previously Gap G-01)
`username`	string	Yes	Service credential
`password`	secret	Yes	Encrypted at rest — never logged (ES03; Previously Gap G-01)
`default_collection_href`	URL	No	Pre-selected collection from Service Document
`accepted_formats`	string	No	Comma-separated extensions and/or MIME types used to build file-input `accept` attributes and validate uploads. See Accepted File Formats
`descriptive_metadata`	boolean	Yes	Per-repository switch for sending descriptive metadata

Save / test actions (BS03): **Test connection** GETs the Service Document, validates auth, and populates the collection list; failures surface to the admin.

Field group: Descriptive Metadata

Selections in this field group will override default metadata settings.
The user clicks a button to add a new entry.
Each new entry has two dropdowns.

1. Source Field

2. Destination Field

Source field options:

- V2 Implementation Gap G-01; Feature implementation team and ArchivesSpace product team to determine a set of ArchivesSpace fields that should be available to map, along with the transformation requirements and constraints.

Destination field options:

`dc.title`
`dc:creator`
`dc:date` / `dc:date.issued`
`dc:description`
`dc:subject`
`dc:identifier`

Configuring descriptive metadata per repository is a complex and error prone design. This spec only states that it should be implemented if possible, and suggests the method of selecting source and destination fields from dropdown menus. The spec does not define sample code or the true development pathway to enabling this part of the feature.

Each destination field may be mapped to only one source field. The implementation team must determine how to implement constraints and errors for ensuring 1:1 field mappings.

Reference UI touchpoint

Mode A — "Upload File Version" (host record: Resource or Archival Object)

The control lives **in the File Versions subrecord section of the Create Digital Object modal**, aligned with **Add File Version** (per the observation that "Add File Version" simply appends an empty File Version row). The File Versions form is shared with the standalone Digital Object edit screen, so the plugin **must scope the injection** to the Resource/AO modal context (A-21) — it is not a global File Versions override.

Flow: select the host record (a Resource, or a child Archival Object) in the tree → Instances → **Add Digital Object** → dropdown → **Create** → the "Create Digital Object" modal opens → scroll

to **File Versions** → **"Upload File Version"** → select **one** file → the Identifier and a File Version row are populated with the returned URI → **"Create and Link"** persists the Digital Object and the instance link to the host record → move to the **next** record and repeat.

Element	Behavior
Entry contexts	**Only** the "Create Digital Object" modal opened from a **Resource or Archival Object** Instances group. **Not** shown on the standalone Digital Object edit screen; **Digital Object Components are out of scope** (A-21 / D-17).
"Upload File Version" button	Rendered next to "Add File Version" in that modal when the repository has SWORD enabled (ES01 hides/disables otherwise, with tooltip).
Hidden file input	`<input type="file" accept=".pdf,.zip">` — **single file** (A-13; no `multiple`). `accept` is illustrative; build it from `accepted_formats`.
Collection select	Shown only if >1 collection or no default; else uses `default_collection_href`; "None" allowed.
Progress + result	Upload progress; on success, **set the Digital Object Identifier** (required, canonical URI) and **populate a File Version row** (`file_uri`, `publish` checked) exactly as if added manually (A-19); on failure, inline error (ES02/ES03/ES04).
Save semantics	The Identifier and the deposited File Version are populated into the in-memory form and persisted on **"Create and Link"** (A-11). No auto-save is required; see D-03.

Mode B — "Upload and Link" panel (host record: Resource or Archival Object)

A plugin-injected panel on the host record (proposed placement: a new section under **Instances**). It renders one row per **immediate child** — top-level Archival Objects for a Resource, child Archival Objects for an AO (A-15/A-20) — enumerated via the tree API (A-16).

Flow: open the host record (Resource or parent Archival Object) → the **"Upload and Link"** panel lists immediate children → pick a local file in each child's input (leave others blank) → click the single **"Upload and Link"** button → the plugin deposits + creates + links per child → a per-child result summary appears with links to each new Digital Object / repository item.

Element	Behavior
Panel visibility	Shown on Resources and Archival Objects that have children when SWORD is enabled for the repository (ES01 otherwise hidden/disabled with tooltip).
Child rows	One row per immediate child: child title/label + a single `<input type="file" accept=".pdf,.zip">` (one file per child, A-15). Children with no file selected are skipped. `accept` is illustrative; build it from `accepted_formats`.
Collection select	One selector for the batch (per-child override deferred); default from config (D-06).
"Upload and Link" button	Submits all chosen `{ child → file }` pairs to `POST /plugins/sword_deposit/deposit_and_link`.
Progress + result	Per-child status (pending → deposited → linked, or error). On success, the row shows a link to the new Digital Object and the repository item; on failure, an inline, retryable error (ES02/ES03/ES04).
Persistence	**Server-side** — the plugin creates each Digital Object and links it to the child AO (A-14). No host-form save is required; results are already persisted when the summary appears.

Mode C — "Upload Digital Objects" button (host record: Resource or Archival Object)

A plugin-injected button in the **Instances** section of the host record (Resource or AO), beside **"Add Digital Object."** It opens a **multi-select** file dialog; each chosen file becomes its own Digital Object linked to the host record.

Flow: open a Resource or Archival Object → scroll to **Instances** → click **"Upload Digital Objects"** → multi-select one or more files → the plugin deposits each via SWORD → for each, a **digital-object instance** appears in Instances with **Title = filename (no extension)**, **Identifier = returned URI**, and **one File Version** (`file_uri` = returned URI) → click **Save** to persist all of them.

Element	Behavior
"Upload Digital Objects" button	Rendered next to "Add Digital Object" in the host record's Instances section when SWORD is enabled (ES01 hides/disables otherwise, with tooltip).
File input	`<input type="file" accept=".pdf,.zip" multiple>` — **multi-select** (A-17). `accept` is illustrative; build it from `accepted_formats`.
Collection select	One selector for the whole selection; default from config (D-06).
Progress + result	Per-file progress; on success, **add a digital-object instance** to the host record form (Title, Identifier, and one File Version pre-filled — A-19); on failure, inline per-file error (ES02/ES03/ES04) with the other files unaffected.
Persistence	The new Digital Objects are populated **in-form** and persisted on the host record's native **Save** (A-18). Multi-nested create should be verified (D-13); server-side pre-create is the fallback.

i18n: add keys under `frontend/locales/en.yml` following AS locale conventions.

Accepted File Formats

The `accept=".pdf,.zip"` values in the Mode A/B/C sketches are *examples* showing how to list multiple types in an HTML file input. Implementations *must* build each mode's `accept` attribute (and server-side upload validation) from the repository's Tier 2 `accepted_formats`

setting (falling back to Tier 1 `AppConfig[:sword_deposit_allowed_mime_types]` when blank).

`accepted_formats` is a comma-separated string of *extensions* (e.g. `.pdf`, `.zip`) and/or *MIME types* (e.g. `application/pdf`, `application/zip`). Values should be a *subset* of what the target IR can ingest. For the DSpace reference deployment, use formats from DSpace's bitstream format registry; do not invent a parallel catalog:

- DSpace bitstream-formats.xml
- Dataverse Registry

Accepted file formats should be configurable by the admin in a GUI; see Tier 2 — AS Administrator (per-repository SWORD settings).

Metadata and packaging

Descriptive metadata for the IR is read from the **persisted ArchivesSpace host record** (or Mode B child AO) as a **hard-coded Dublin Core subset** and attached to the deposit, the same pattern as V1's PDF path (hard-coded DC on the file deposit), without a Java SWORD client or METS packaging option. Allowed file types are configured per repository (Accepted File Formats); they are not limited to PDF.

Approach

- **Deposit type: binary** — a single file `POST` to the Collection IRI (`Content-Type` from the file, `Content-Disposition` carrying the filename), plus an Atom entry carrying Dublin Core from `HostRecordDcMapper`. No packaging URI. Formats follow `accepted_formats` (e.g., PDF, ZIP as ordinary bitstreams).
- **Hard-coded AS → DC map:** the plugin reads a **small code-based set** of fields from the host/child record and passes a **map of DC field → AS value** into `SwordDepositController` / `SwordDepositService`. Sample code shows **title only**; other mapped fields follow the same pattern. This is **not** an Admin field-mapping UI — adding fields means editing `HostRecordDcMapper`.
- **Mode C title exception:** for multi-file deposits onto one host record, **`dc:title` and the AS Digital Object Title** both use **filename without extension** (A-17) so IR items / Digital Objects are not identically named after the host. All **other** DC fields still come from the host record. Modes A/B take `dc:title` from the host/child record.
- **Starting reference:** as-dc-mapping informed the subset below but was authored for **export**, not SWORD deposit. Confirm field-by-field with stakeholders before expanding (D-18).

AS → Dublin Core mapping

For every deposit, ArchivesSpace reads a **small hard-coded set** of properties from the selected host record (Modes A/C) or target child Archival Object (Mode B) and attaches them to the SWORD deposit as Dublin Core. Direction: **from AS to DC for deposit**. **Mode C** overrides `dc:title` with each file's basename; other DC fields still come from the host.

Source data:

- DSpace
- ASpace

ArchivesSpace Source	Dublin Core (deposit)	Notes
Resource / Archival Object **Title** (`title`, fallback `display_string`)	`dc.title`	Primary sample field for Modes **A/B**; see `HostRecordDcMapper`. **Mode C:** use **file basename without extension** instead (A-17) so multi-file deposits are distinct.
`` `linked_agents` `` where role == creator and relator is not `` `ctb` `` or `` `pbl` ``	`dc:creator`	Flatten agent display names to one or more `dc:creator` values when present
`dates`: use `expression` if present, else `begin` + `"..."` + `end`	`dc:date` / `dc:date.issued`	DSpace often treats this as Publication Date; exact string format is Implementation Gap G-03
`notes` where type = `scopecontent`	`dc:description`	Omit when absent
`linked_agents` where role == subject	`dc:subject`	Optional
Public proxy URI of the host record (`AppConfig[:public_proxy_url]` + record uri)	`dc:identifier`	Optional; distinct from the **returned** IR item URI written back to the Digital Object

Configuring descriptive metadata (admin)

Sending descriptive metadata with a SWORD deposit is **not** a hard requirement. Administrators shall be able to turn descriptive metadata for SWORD deposits **on or off**. If the default mapping

(DSpace-oriented) does not meet a non-DSpace endpoint's needs, turning it off keeps the deposit working.

Administrators will **not** be able to add new mapped fields unless they edit the source code (same stance as V1).

Implementation notes (not all shown in sample code):

- Omit a DC element when the AS source value is absent rather than sending an empty element.
- Multi-valued AS fields (e.g. creators) may yield repeated DC elements; do not invent an Admin UI to configure this in this version.
- Mode C: build `dc_base = HostRecordDcMapper.from(host_ref)`, then per file `dc_fields = dc_base.merge("dc:title" => File.basename(filename, ".*"))`.
- A fuller IR-specific mapping remains (v2 Implementation **Gap G-01**); this version implements this **minimal DC subset** only.

AS record fields written per mode

Every mode writes the returned URI to **both** Digital Object fields (A-19): the required **Identifier** (canonical URI) and a **File Version** `file_uri` (clickable PUI link). The **Title** column below is the ArchivesSpace Digital Object title. For Mode C, the same basename is also SWORD `dc:title`; for Modes A/B, SWORD `dc:title` comes from the host/child via `HostRecordDcMapper` (A-12).

Mode	AS record	Title source (AS DO)	Identifier (`digital_object_id`)	File Version (`file_uri`)
A	Digital Object created in the modal	in-form Title	returned URI (required)	returned URI (A-07)
B	New Digital Object per child	child AO `display_string`	returned URI (required)	returned URI (A-07)
C	New Digital Object per file	**filename without extension** (A-17; also SWORD `dc:title`)	returned URI (required)	returned URI (A-07)

D-14 (resolved 2026-07-20): write **both**. The Digital Object **Identifier** (`digital_object_id`) is a **required** field and holds the canonical/original URI; a **File Version** `file_uri` carries the same URI so the PUI renders a clickable "view online" link (sandbox-verified: Identifier-only shows plain text; a File Version shows a clickable button). This also keeps File Versions available for the future re-deposit / new-upload scenario (v2 Implementation Gap G-02).

Packaging mode behind the adapter

Mode	When	Notes
binary (this version)	Collection accepts the file MIME / format	Bitstream + hard-coded Atom/DC from host record; formats from `accepted_formats`; only mode in this revision

SWORD v2 protocol surface (implementation mapping)

Operation	SWORD v2	Adapter method
Service Document	`GET` Service-Document IRI (Basic auth)	`service_document`
List collections	Parse workspaces from Service Document	(from `service_document`)
Deposit	`POST` file to Collection IRI	`deposit` → `DepositResult`
In-progress / complete	`In-Progress` header; Status/Edit-IRI in receipt	`status` (optional v0.1)
Auth	HTTP Basic	credential from config

Headers: **Authorization**, **Content-Type**, **Content-Length**, **Content-Disposition**, optional **Slug**, **In-Progress**, **Packaging**.

Which IRIs to keep: the receipt yields several IRIs. Store the **public item URL** in **both** the Digital Object **Identifier** (`digital_object_id`, required) and a **File Version** `file_uri` (A-07/A-19); optionally retain the **Edit-IRI** in the audit log to enable future re-deposit/update (v2 Implementation Gap G-02).

Client library note (`sword2ruby`)

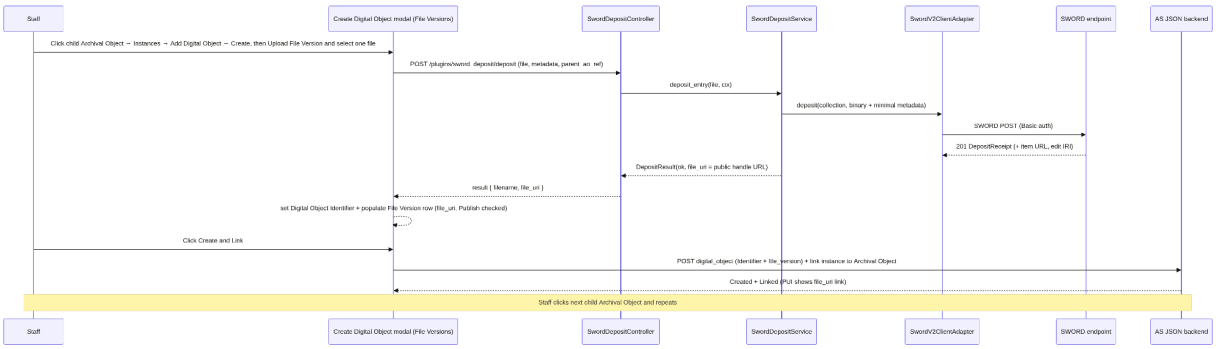
`sword2ruby` is a genuine SWORD v2 Ruby client but appears **effectively unmaintained** (SWORD-project origin, ~2012-era, `atom-tools`-based, minimal repo activity). Recommendation: treat it as a **reference implementation** behind `SwordProtocolAdapter`, and default `AppConfig[:sword_deposit_client]` to a **thin native `Net::HTTP` client** for v0.1 (a v2 binary deposit is a small, well-specified HTTP interaction). This isolates the dependency decision to one class and keeps upgrade/replacement cheap.

SWORD v3 forward compatibility

Concern	v0.1 approach
Adapter selection	`SwordAdapterFactory.for(cfg.protocol_version)`
v3 implementation	`SwordV3ClientAdapter` raises `NotImplementedError`
Auth	Config schema reserves OAuth/bearer fields; unused in v0.1

Data flows

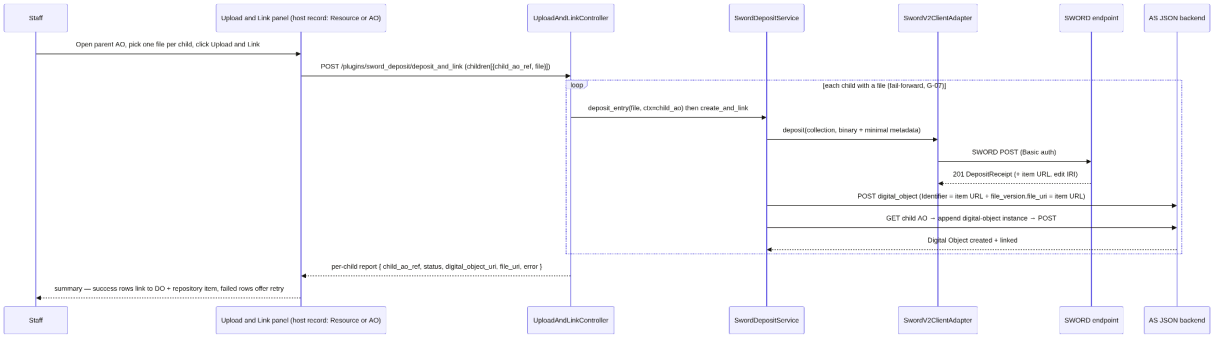
Mode A (single deposit, in-form population)



Note over S,B: Staff clicks next child Archival Object and repeats

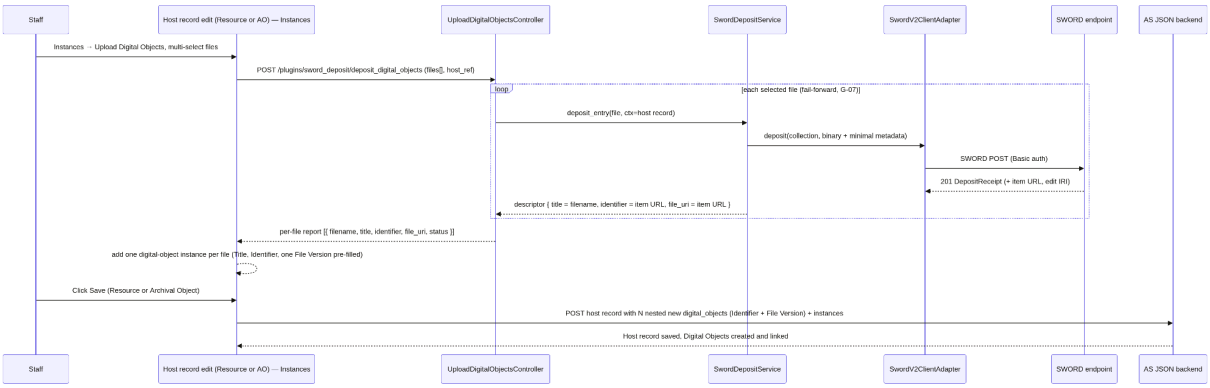
Mode A performs **no AS write** during the deposit; it returns the `file_uri` for in-form population (A-11). AS persistence — creating the Digital Object (with its required Identifier and a File Version, A-19) and the instance link back to the host record — happens on the native **"Create and Link"** submit. The staff user then selects the next host record in the tree and repeats (A-13). The deposit control is **not** offered on the standalone Digital Object edit screen (A-21).

Mode B (batch "Upload and Link")



Mode B **writes to AS server-side** (A-14): for each child with a file it deposits, then creates a Digital Object and links it to that child. There is no parent-form save step — results are persisted as the summary renders. Failures are isolated per child (fail-forward); a deposit that succeeds while the AS write fails leaves a repository item whose Edit-IRI is logged for cleanup (D-12).

Data flow: Mode C (multi-DO "Upload Digital Objects")



Mode C **deposits many files** but writes like Mode A: it returns one descriptor per file and the browser adds a digital-object instance to the open host-record form (Title = filename, Identifier = URI, and one File Version `file_uri` = URI — A-19). AS creates the new Digital Objects and links them on the host record's native **Save** (A-18). Multi-nested create is the item to verify (D-13); if unsupported, fall back to server-side pre-create per file (ES05 orphan caveat applies).

Behavior Scenarios

BS-01: Administrator configures a SWORD deposit endpoint

Step	Description
Given	The user has Administrator-level access to ArchivesSpace.
	The administrator has a deposit account for a SWORD v2 enabled repository.
When	The administrator navigates to System > Manage Repositories, selects the repository, and opens SWORD Deposit Settings.
	The administrator enters the required fields (Configuration Fields (Proposed)
Then	The system retrieves and validates the SWORD Service Document from the configured endpoint URL.
	The system notifies the administrator that the SWORD endpoint is enabled.
	The SWORD Deposit button becomes active in the SUI for staff users of that repository.

BS02: Staff user sets up a deposit for a batch of files that relate to many Archival objects

Step	Description
Given	A Resource with Archival Objects or an Archival Object with child archival objects exists in ArchivesSpace.
	Multiple files are associated with multiple archival objects.
	SWORD deposit is configured and enabled.
When	From a Resource or Archival Object edit page, the staff user navigates to a new field group, e.g. “Batch Upload and Link”, and clicks a button to add a new batch upload.
Then	Two panels load as columns within the field group. A list of all the Resource’s children AO’s or the AO’s children AO’s are listed in Panel 1 (left side). Panel 2 (right side) is a file input box for each AO.
When	The staff user selects a file input box
Then	A file input dialog appears
When	The staff user selects the file to upload and relate to the corresponding digital object, and then clicks “Open”
Then	The file is selected in Panel 2.
And	The staff user continues to add a file to each AO in this manner.

Notes

The deposit is not **linked** to the host record until the user selects “Create and Link” (See BS05)

BS03: Staff user sets up a deposit for a batch of files that relate to one Resource or Archival Object

Step	Description
Given	A Resource or Archival Object exists in ArchivesSpace.
	Multiple files are associated with one Resource or Archival Object.
	SWORD deposit for ArchivesSpace is configured and enabled.
When	From a Resource or Archival Object edit page, the staff user navigates to Instances, where there is a new button, e.g. “Upload Digital Objects”, and clicks the button.

Step	Description
Then	A file upload box appears
When	The user selects the upload button on the file upload box
Then	A file upload dialog appears showing the user's file system
When	The user selects multiple files to upload and clicks "Open"
Then	The list of opened files appears in the panel.
And	The deposit is not initiated until the user selects a "Save" button (See BS05)

Notes

The deposit is not **linked** to the host record until the user selects "Create and Link" (See BS05)

BS04: Staff user deposits a single file

Step	Description
Given	A Resource or Archival Object exists in ArchivesSpace.
	One file is associated with one Resource or Archival Object.
	SWORD deposit for ArchivesSpace is configured and enabled.
When	From a Resource or Archival Object edit page, the staff user navigates to Instances, and clicks "Add Digital Object" (existing functionality).
Then	The Digital Object selection box appears. (existing functionality)
When	The user clicks the digital object action menu (triangle button) and selects "Create" (existing functionality)
Then	The Create and Link data entry form pops up. There is a NEW button in the File Versions field group named, e.g., "Upload File Version"
When	The user clicks "Upload File Version"
Then	A file upload dialog appears showing the user's file system
When	The user selects **one file** to upload and clicks "Open"
Then	The plugin deposits the single file via SWORD
And	The plugin populates the File Version's File URI with the result of the SWORD deposit.
And	The plugin also populates the Identifier of the new (unsaved) Digital Object.

Notes

The deposit is not **linked** to the host record until the user selects “Create and Link” (See BS05)

BS05: ArchivesSpace deposits the files, retrieves URIs, and generates digital objects

Step	Description
Given	SWORD deposit for ArchivesSpace is configured and enabled.
	Multiple files are associated with multiple archival objects.
When	The staff user completes the save action via a button.
Then	A SWORD deposit is made for each archival object’s file grouping (sometimes a single file, sometimes a group of files). A SWORD receipt is generated.
When	ArchivesSpace receives the SWORD receipt with new URIs for digital objects in the SWORD-enabled system.
Then	Digital Object records are created in ArchivesSpace for each deposited file, linked to its related Archival Object.
	A batch deposit summary is displayed alongside or utilizing existing object save results output mechanisms in ArchivesSpace.
	Successful deposits are logged; failed deposits are flagged for retry (see ES02).

Error Scenarios

ID	Condition	User-visible behavior	Log / notes
ES01	SWORD not configured/enabled for repository	"Upload File Version" hidden or disabled with tooltip "SWORD deposit is not configured…"	—
ES02	Endpoint HTTP 4xx/5xx or SWORD Error Document	Plain-language error per file; **no File Version** created for that file	Full SWORD body + timestamp + user in audit log; retry/cancel offered

ID	Condition	User-visible behavior	Log / notes
ES03	Auth failure (401/403)	"Authentication failed. Your SWORD credentials may be expired or incorrect. Contact your administrator."	No deposit attempted; **never log password**
ES04	Missing required metadata (e.g. no host-record title for `dc:title`)	Block the file; prompt to fill/acknowledge required fields on the host record	Required DC fields follow AS → Dublin Core mapping (`dc:title` at minimum when metadata is enabled)
—	**Partial batch failure** (previously Gap G-07, Mode B)	**Fail-forward:** children that deposited + created + linked show success; failed children keep their file input for retry, others are unaffected	Consistent with A1-2 `LinkBatch` semantics; per-child result in the report
ES05	**Deposit succeeded but AS create/link failed** (Mode B)	Report the child as failed with a clear message; offer retry	**Repository item is orphaned**; log the Edit-IRI for cleanup. Default is fail-forward, not rollback (D-12)

Other Requirements

Documentation

Documentation for this feature should include:

- How to configure your repository and enable SWORD
- How to deposit files or groups of files using the ArchivesSpace SWORD app
- Data requirements for successful SWORD deposits

    - Re-defining accepted formats from the default, per endpoint repository's requirements
- Troubleshooting
- Governance, support, and contact information
- Accessibility features and compliance
- Reference documentation, such as field definitions

The documentation may include screenshots and examples.

Governance

The feature design should be reviewed, further edited, and if agreed upon, formally accepted by the ArchivesSpace development team.

Maintenance of this feature after the implementation phase will fall to the ArchivesSpace development team.

Error Reporting

Errors are reported in ArchivesSpace server logs.

Performance and Scalability

This plugin adds a staff-initiated deposit path; it does not introduce background harvest traffic, global search load, or long-running AS-side processing. Impact on ArchivesSpace performance and scalability should remain modest.

- **Transit path.** Uploaded files travel through ArchivesSpace (the staff frontend plugin holding the authenticated session) to the Institutional Repository (IR) over SWORD. Binaries are held only transiently in AS (A-04/A-05); AS does not store deposited files as its own bitstreams.
- **Throughput.** Deposit rates track **individual staff usage** and client/network **upload speeds**, not a shared AS indexing or harvest pipeline. Concurrent deposits are bounded by how many staff users deposit at once.
- **Mode B batching.** Mode B may deposit several children in one action. Implementations **may** sequence or otherwise optimize those uploads asynchronously relative to browser and ArchivesSpace server sessions so a long batch does not monopolize a single request. Fail-forward reporting still applies.
- **IR In-Progress.** If DSpace (or another IR) returns SWORD **In-Progress** because of load or slow ingestion on its side, that status does **not** degrade ArchivesSpace operation: AS records the returned URI and continues (A-09 / D-07). AS does not wait on IR-side ingest completion.

- **Overall.** The feature should **not dramatically affect** ArchivesSpace performance or scalability under normal staff-deposit workloads. Capacity planning remains dominated by file size, network path to the IR, and IR ingest behavior rather than by AS core services.

Decisions: Implementation V1

The consultant spec designers made the following decisions throughout the design process, which could be reversed or reviewed further with the implementation team:

ID	Question	Options	Default recommendation
D-01	Client library	`sword2ruby` vs native `Net::HTTP`	**Native client**, `sword2ruby` as reference
D-02	Config store	Repository preference vs plugin backend model vs JSON file	**Plugin-managed per-repository record**
D-03	When File Versions persist	On record Save/"Create and Link" vs immediate auto-save on deposit	**Persist on Save / "Create and Link"** — *required* for the primary AO flow, where the Digital Object is unsaved at deposit time (A-11); auto-save is only viable for a pre-existing Digital Object
D-04	Credential storage	Encrypted field vs external secret	**Encrypted at rest**; document production hardening (previously Gap G-01)
D-05	Orchestration tier	Frontend plugin vs backend job	**Frontend plugin** (session reuse; binaries avoid JSON backend)
D-06	Collection selection UX	Always show vs default+hide	**Default when configured; show when ambiguous**
D-07	In-progress deposits	Hold vs write immediately	**Write File Version immediately** (A-09)
D-08	Re-deposit/update	New item vs SWORD replace	**New item (deposit-only)** in v0.1 (see v2 Implementation Gap G-02)
D-09	**Mode B panel placement**	New section under **Instances** on the host	**Section under Instances** on the host record — Resource or AO

ID	Question	Options	Default recommendation
		record vs a separate panel/tab	(matches the client's "under Instances" ask); confirm with UX
D-10	**Mode B write path**	Server-side create + link vs client form population	**Server-side create + link** (A-14) — required because Mode B spans sibling records with no single open form
D-11	**Mode B child scope**	Immediate children only vs recursive/deep	**Immediate (direct) children only** in v0.4 (A-15/A-20: Resource → top-level AOs, AO → child AOs); deep selection deferred to the wizard
D-12	**Mode B transactionality**	Fail-forward per child vs all-or-nothing vs rollback of orphaned items	**Fail-forward per child** (previously Gap G-07); log Edit-IRI of orphaned items (ES05); auto-rollback deferred
D-13	**Mode C multi-nested create on one save**	In-form nested create of N Digital Objects on the host record's Save vs server-side pre-create per file	**In-form nested create** (A-18) — orphan-free and consistent with Mode A; **verify in the sandbox** that the Resource/AO form serializes multiple nested new Digital Objects, else fall back to server-side pre-create
D-14	**Where the returned URI lands (all modes)**	Identifier only vs File Version only vs **both**	**Both (resolved 2026-07-20):** Identifier (`digital_object_id`, *required*) = canonical URI **and** a File Version `file_uri` = same URI for the PUI clickable link (sandbox-verified). Applies to Modes A, B, and C (A-19)
D-15	**Mode C button placement/label**	"Upload Digital Objects" beside "Add Digital Object" vs elsewhere in Instances	**Beside "Add Digital Object"** in the Instances section (matches the client ask); confirm label/UX
D-16	**Host record types**	Archival Objects only vs **Resources and Archival Objects**	**Both (client ask, sandbox-confirmed 2026-07-20):** start from a Resource or an Archival

ID	Question	Options	Default recommendation
			Object; identical Instances UI/tree (A-20)
D-17	**Visibility on Digital Objects / Components**	Also inject on standalone DO (and DOC) edit screens vs **Resource/AO only**	**Resource/AO only (client ask, 2026-07-20):** do **not** show deposit controls on the standalone Digital Object edit view; **exclude Digital Object Components entirely**. Implementation must scope the File Versions injection to the host-record Create Digital Object modal (A-21)
D-18	**Descriptive metadata mapping AS → repository**	binary-only vs hard-coded DC subset vs Admin mapping UI; and *which* AS record supplies it	See v2 Implementation Gap G-01 for v2 recommendations. For V1: **Hard-coded minimal AS → DC subset** via `HostRecordDcMapper` from the persisted **host record** (Modes A/C) or **child AO** (Mode B) — A-12; sample code shows `dc:title` only; **Mode C overrides `dc:title` with file basename** (A-17); Admin on/off toggle for sending DC; no field-mapping UI; expand mapping later with AS+DSpace teams (v2 Implementation G-01). METS deferred; binary formats via `accepted_formats`.

Implementation Gaps

Resolved Specification Gaps between v0.3 and v.0.12:
Resolved Gaps between v0.3 and v0.12 for A3-4 spec

#	Gap	Description	Owner
G-01	Configurable descriptive metadata	There is no 1:1 mapping of ArchivesSpace to Dublin Core. Most ASpace data requires significant transformation in DC mapping. The recommendation for v1 implementation is to hard-code select fields to DC. A future implementation could make this configurable, but two important gaps would need to be addressed: 1. A selection of ArchivesSpace fields for the user to choose from for mapping to DC 2. Unique code for each selected field that defines how to transform the data for DC 3. Validation or constraints that prevent multiple ASpace fields to be mapped to one DC field; or concatenation code if this is to be allowed. The v1 behavior described in this spec is: **Hard-coded minimal AS → DC subset** via `HostRecordDcMapper` from the persisted **host record** (Modes A/C) or **child AO** (Mode B) — A-12; sample code shows `dc:title` only; **Mode C overrides `dc:title` with file basename** (A-17); Admin on/off toggle for sending DC; no field-mapping UI; expand mapping later with AS+DSpace teams. METS out of scope.	V2 Implementation team
G-02	Re-deposit / update-in-place of previously	The v1 spec defines **deposit** only. SWORD has an update protocol, but this would require an entirely new spec doc with additional	V2 Implementation Team

#	Gap	Description	Owner
	deposited content	configuration requirements. Update-in-place is a v2+ implementation gap. The v1 behavior described in this spec is: If a file needs to be updated, the user would submit a new deposit, then manually delete the outdated content in both ASpace and the endpoint repository.	
G-03	Exact date transformation	The v1 implementation team should determine the best way to transform ArchivesSpace date to DSpace and Dataverse repositories. The spec defines a format based on ArchivesSpace DC export rules.	V1 Implementation Team
G-04	Review DSpace Collection requirement against SWORD protocol requirements	Consultants interpreted SWORD protocol as requiring a Collection, so we included a requirement for naming a DSpace Collection in the spec. Many DSpace items are not in Collections. The SWORD “Collection” requirement should be reviewed in case the consultants interpreted this requirement too narrowly, and collection could be e.g. simply the DSpace endpoint.	V1 Implementation team, Product team
G-05	ArchivesSpace digital_object_ID mapping	Review digital_object_id mapping. The returned DSpace URI is written to two Digital Object fields, per the spec. (`digital_object_id`) — a **required** field on Digital Objects (so it must be set at create time regardless), holding the canonical/original deposit URI; and (2) a **File Version** `file_uri` (public item URL, A-07).	V1 Implementation team, Product team

#	Gap	Description	Owner
		The consultant team acknowledges that a different piece of metadata may be more appropriate for digital_object_id than the returned DSpace URL. The options for populating this field are: (1) inherit data from a field in the archival object; (2) populate with data from SWORD (current solution). (2) is the current solution as written in the spec: populate digital_object_id with SWORD location (DSpace URI). Other data from SWORD includes status, and other link types that are typically not user-facing.	

Suggested epics

Epic	Deliverable	Area
0 — Spike	Native v2 binary deposit against a test DSpace SWORD endpoint; parse receipt → item URL	`lib/sword`
1 — Config	Per-repository SWORD settings + Test connection + enable flag	frontend/backend config
2 — Upload UI (Mode A)	"Upload File Version" control (single file), progress, in-form row + Identifier population; injected into the File Versions subrecord **only** in the Resource/AO "Create Digital Object" modal (scoped away from standalone DO / DOC — A-21)	frontend assets/views
3 — Deposit + in-form population (Mode A)	`DepositEntry` (single file), `HostRecordDcMapper` → `dc_fields`, return `file_uri`; populate **Identifier + File Version**	`lib`

Epic	Deliverable	Area
	(A-19); persistence via native Save / "Create and Link"	
4 — Batch Upload-and-Link (Mode B)	"Upload and Link" panel listing immediate children of a Resource or AO (tree API); `deposit_and_link_batch` + `DigitalObjectLinker` (create DO with Identifier + File Version, link per child DC from child AO per-child report; fail-forward	frontend + `lib`
5 — Multi-DO Upload (Mode C)	"Upload Digital Objects" button (multi-select) in the Resource/AO Instances; `deposit_digital_objects_batch` + host-record DC with **per-file `dc:title`/AS Title = basename** + `DigitalObjectInstanceBuilder`; in-form instance population; verify multi-nested create (D-13)	frontend + `lib`
6 — Hardening	ES01–ES05, partial-batch fail-forward, orphaned-item logging, audit log, credential encryption	plugin-wide
7 — Metadata	Implement `HostRecordDcMapper` fields beyond title; admin on/off for DC; confirm date/creator mapping with AS+DSpace teams; wire `accepted_formats` to UI + validation	`lib/metadata`
8 — Wizard (deferred)	Deep/drag-drop file→archival-object mapping over the same `DepositBatch` + create-and-link machinery	frontend
9 — SWORD v3 (future)	`SwordV3ClientAdapter`, OAuth	`lib/sword`

Related documents

Document	Relationship
June 29 version of this doc	Requirements baseline (BS01–BS03, ES01–ES04, Previously G-01–G-10)
V1	Sibling SWORD deposit design (VIVO/Java); adapter + config patterns reused
A1-2	AS write path (no PATCH; find→merge→save); `Entry`/`Batch` precedent
as-dc-mapping	Candidate AS → Dublin Core mapping (export-era; needs review for deposit)
swordapp/sword2ruby	Candidate Ruby SWORD v2 client (maintenance caveat)

Document history

Version	Date	Notes
0.1-draft	2026-07-17	Initial high-level design: AS plugin map, "Upload File Version" multi-select control, `DepositEntry`/`DepositBatch` (1:1:1), public-handle `file_uri`, v2 adapter (native vs `sword2ruby`), immediate File Version write with Publish/Representative deferred to staff, metadata mapping as M-01 placeholder. Wizard deferred.
0.2-draft	2026-07-19	Made the **Archival Object** the primary entry point (Instances → Add Digital Object → Create → File Versions), verified against the ArchivesSpace sandbox. Reframed write-back as **in-form File Version population** persisted on "Create and Link" / Save (A-11), since the modal's Digital Object is unsaved at deposit time; server-side JSONModel write demoted to an optional path for pre-existing records. Deposit no longer requires a saved target; metadata now sourced from in-form values and/or the parent Archival Object (A-12). Updated purpose, scope, actors, URL shapes, reuse map, UI touchpoints, data flow, decisions (D-03, M-01), and epics.

Version	Date	Notes
0.3-draft	2026-07-19	Simplified to match user studies: **one binary per child Archival Object**, repeated across the tree. **Dropped multi-select**; single-file upload only (`DepositEntry`; `DepositBatch` retained solely for the deferred wizard). Button label singular ("Upload File Version"). Added the observed step-by-step workflow and A-13 (AOs always pre-exist; per-child iteration is native tree navigation). Updated purpose/scope, in/out-of-scope, design stance, actors, controller/service and JS sketches, UI touchpoints, and data-flow diagram.
0.4-draft	2026-07-19	Added **Mode B — "Upload and Link"**: a parent-Archival-Object panel listing immediate children with one file input each, and a single action that deposits + creates + links a Digital Object per child. Framed the design as **two modes** sharing the `DepositEntry` primitive: Mode A (in-form population, unchanged) and Mode B (**server-side create + link**, realizing `DepositBatch`). Added A-14/A-15/A-16 (server-side write, immediate children, tree-API enumeration), `UploadAndLinkController` / `deposit_and_link_batch` / `DigitalObjectLinker` / `ArchivalObjectChildren` components, Mode B UI section, a second data-flow diagram, ES05 (orphaned item), and D-09–D-12. Distinguished Mode B from the still-deferred deep/drag-drop wizard.
0.5-draft	2026-07-20	Combine architecture-forward design with behavior scenarios and draft design
0.6-draft	2026-07-20	Remove references to creating a new **File Version**, favoring the digital object basic info for record creation Include **Resource** record types as parent record options **Add Path C** - add multiple digital objects to **one** archival object or resource
0.7-draft	2026-07-20	**Generalized the starting point to a Resource *or* an Archival Object** (client ask). Introduced the **"host record"** concept: both record types expose the same Instances group and tree and can carry digital-object instances, so all three modes work identically from either (A-20, D-16). The only type-dependent behavior is Mode B's *immediate children* (Resource → top-level AOs; AO →

Version	Date	Notes
		child AOs — A-15). Threaded "host record" through the three-modes table, Mode A/B/C narratives and UI sections, actors, URL shapes (added Resource edit + resource tree root), areas-touched (`resource`/`archival_object` JSONModel), reuse map, controller/service sketches (`host_ref`), and all three data-flow diagrams. Noted DO/DOC remain Mode A subrecord-only entry points.
0.8-draft	2026-07-20	**Confined the feature to Resource and Archival Object edit views** (client ask). Deposit controls are **not** shown on the standalone Digital Object edit screen; **Digital Object Components are out of scope entirely** (A-21 / D-17). Mode A's "Upload File Version" injection is scoped to the Resource/AO Create Digital Object modal (shared File Versions form requires an explicit context gate). **Sandbox-confirmed** that a Resource has the same Instances options and shows AO children in the top tree (A-20). Updated purpose, in/out-of-scope, UI entry contexts, data-flow notes, epics, and decisions.
0.9-draft	2026-07-30	**Hard-coded AS → Dublin Core** (aligned with V1 pattern): `HostRecordDcMapper` builds a DC field → AS value map from the persisted host/child record and passes it into `SwordDepositController` / `deposit_entry` (sample shows `dc:title` only). **Mode C:** `dc:title` and AS Digital Object Title use **per-file basename**; all other DC fields still come from the host record. METS / packaged deposit deferred (RoR client limitation). Updated A-10/A-12/A-17, class sketches, metadata section, and epics. M-01 replaced by AS → Dublin Core mapping
1.0-draft	2026-08-10	**Any configured binary format** (not PDF-only): Tier 2 `accepted_formats`; Mode A/B/C file inputs use illustrative `accept=".pdf,.zip"` built from config; Accepted File Formats links DSpace `bitstream-formats.xml`. Renamed `DepositPackage.pdf` → `DepositPackage.build`. METS **packaging** is still out of scope; `.zip` as an ordinary binary bitstream is allowed when configured.